设计报告

钟志昊 3230105732

目录

一、 文档介绍 1
1.1 编写目的 1
1.2 文档范围 1
1.3 读者对象 1
二、 项目介绍 2
2.1 项目说明 2
2.2 项目背景 2
2.3 需求概述 2
三、 总体设计 4
3.1 技术介绍 4
3.2 架构图 5
3.3 系统运行环境 7
四、 数据库设计与 ER 图 8
4.1 数据库表设计 8
4.2 ER 图 10
五、 系统接口设计 10
5.1 统一说明 10
5.2 用户信息相关接口 10
5.3 图片类接口 15
六、 界面原型 22
6.1 注册界面 22
6.2 登录界面 22
6.3 网站首页 23
6.4 上传图片界面 23
6.5 查看图片界面 24
七、 系统出错设计 27
7.1 出错信息 27
7.2 补救措施 28
八、 附录 28
8.1 项目进度安排 28
8.2 备注 28

此处用于插入实验报告信息

一、文档介绍

1.1 编写目的

本软件产品功能设计说明书（SRS）旨在实现以下目标:

1. 明确定义软件系统的总体要求，为开发阶段的设计和编码工作提供坚实基础
2. 为软件系统的全面测试提供可靠的依据和标准
3. 为系统后续的维护和升级工作提供清晰、完整的参考文档
4. 为系统验收工作提供明确的判断标准，确保系统符合预期要求
5. 建立完善的需求变更管理体系，确保所有变更都有据可查

1.2 文档范围

本图片管理网站软件产品设计说明书全面描述了系统的功能特性及操作方式，详细阐述了系统的架构设计、接口设计、界面设计和数据库设计等核心内容，并提供了系统模块图、系统架构图和 ER 图等可视化设计文档。

1.3 读者对象

本设计说明书面向以下读者群体:

1. 项目客户和利益相关者
2. 需求分析人员
3. 系统开发人员
4. 软件测试人员
5. 系统维护人员
6. 项目管理人员

二、项目介绍

2.1 项目说明

本项目是浙江大学 2025-2026 秋冬学期《B/S 体系软件设计》课程的实践项目，旨在构建一个功能完善的图片管理网站。该系统支持用户注册登录后上传个人图片，并提供分类、检索、编辑、删除和查看等完整的图片管理功能。网站界面采用响应式设计，能够良好适配 PC 端和移动端设备。此外，系统还预留了 AI 图像识别和 AI 对话搜索等先进功能的扩展接口。项目要求提供完整的技术文档，包括设计报告、测试报告和用户手册等。

2.2 项目背景

随着智能手机、数码相机和社交媒体的快速发展，个人用户在日常生活中的数字图片和照片数量呈指数级增长。这些图片承载着用户珍贵的回忆、重要的工作资料或创作素材，具有重要的保存价值。然而，现有的图片管理解决方案普遍存在以下问题：

1. 数据分散与同步效率低下：图片文件分散存储在手机、电脑、云服务等多个平台，缺乏统一、集中的管理入口，导致查找困难且容易造成重复存储。
2. 手动分类与组织成本高昂：传统的管理方式（如文件夹分类）严重依赖用户手动操作。面对海量图片，手动整理分类不仅耗时巨大、容易出错，而且往往无法充分利用图片本身包含的丰富元数据信息。
3. AI 技术应用不足：目前市面上的图片管理网站普遍缺乏对 AI 技术的深度集成。随着 AI 技术的不断发展，已经可以实现使用 AI 进行图片分类和智能搜索，这将极大提升图片管理的效率和准确性。

2.3 需求概述

2.3.1 功能需求

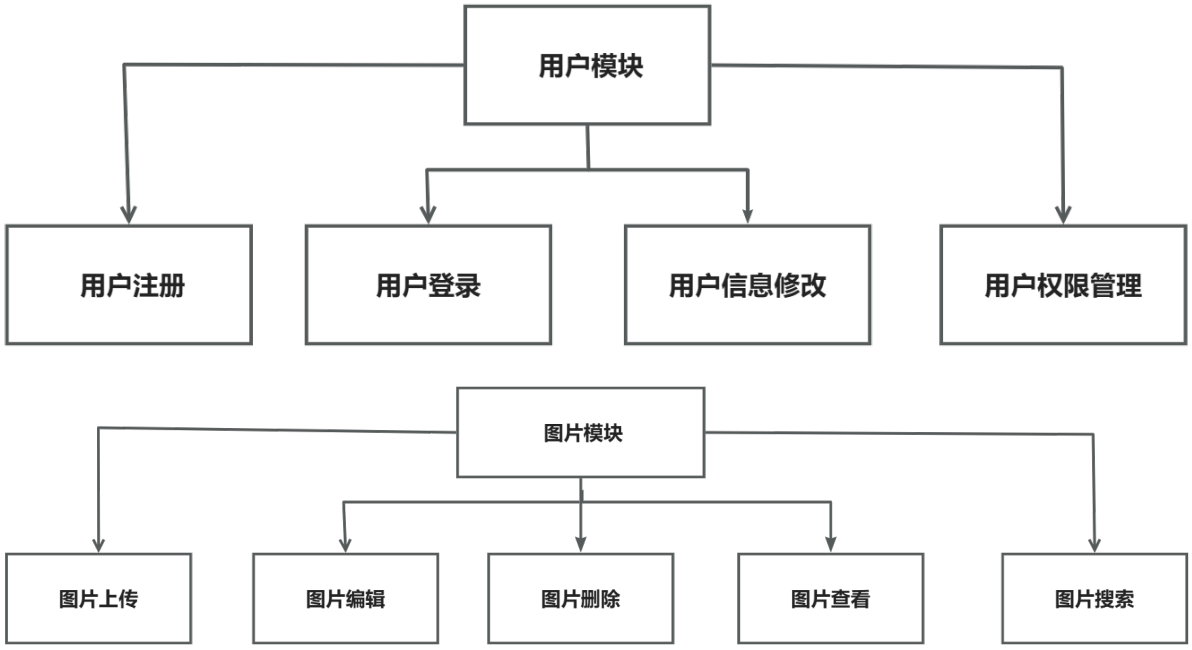
本项目是一个基于 B/S 架构的软件系统，根据需求文档的要求，需要实现以下核心功能：

1. 用户注册（需要用户名、密码、邮箱等基本信息）
2. 用户登录认证
3. 用户信息修改
4. 图片上传（支持自动提取信息和手动设置标签）
5. 图片管理（包括编辑、删除、查看等操作）
6. 缩略图自动生成和展示
7. 多条件组合检索图片
8. 支持图片轮播等丰富的展示方式

此外，系统还支持以下扩展功能：

1. AI 图像识别
2. AI 对话搜索图片

该系统主要面向两类用户：**普通用户**和**管理员**。普通用户只能查看和管理自己上传的图片，而管理员可以查看和管理所有用户的图片，以便进行系统监管。因此，项目的功能可以分为两大核心模块：用户管理模块和图片管理模块，如下图所示：



其中每个功能都有多种具体的实现方式，例如自动添加标签可以从图片的 EXIF 信息中自动提取，也可以由 AI 智能识别；图片查看可以是单张图片查看，也可以是图片轮播展示等。详细的技术实现方案将在后续章节中逐一介绍。

2.3.2 性能需求

系统应具备良好的整体性能表现，确保在实际使用过程中为用户提供流畅、稳定的体验。具体要求如下：

1. 响应速度：页面加载时间控制在 3 秒以内，用户操作响应时间不超过 5 秒
2. 并发处理：具备一定的并发访问能力，支持多用户同时上传和查看图片
3. 浏览器兼容性：兼容主流浏览器，包括但不限于 PC 端和移动端的 Firefox、Chrome、Safari、Edge 等，以及微信等应用的内置浏览器
4. 数据准确性：确保前端展示的数据能够实时同步后端数据库的最新信息，保证数据的一致性和准确性

2.3.3 安全性需求

考虑到用户上传的图片可能涉及个人隐私，系统必须具备完善的安全防护机制。具体要求如下：

1. 密码安全：要求密码长度不少于 6 位，在数据库中存储加盐哈希值，防止密码泄露
2. 攻击防护：防范 SQL 注入等常见网络攻击，设置密码错误尝试次数限制
3. 会话管理：对于长时间无操作的登录会话，系统应自动强制登出，防止账户被盗用
4. 输入验证：对前端用户输入进行严格验证，拦截非法字符和恶意代码

2.3.4 可维护性需求

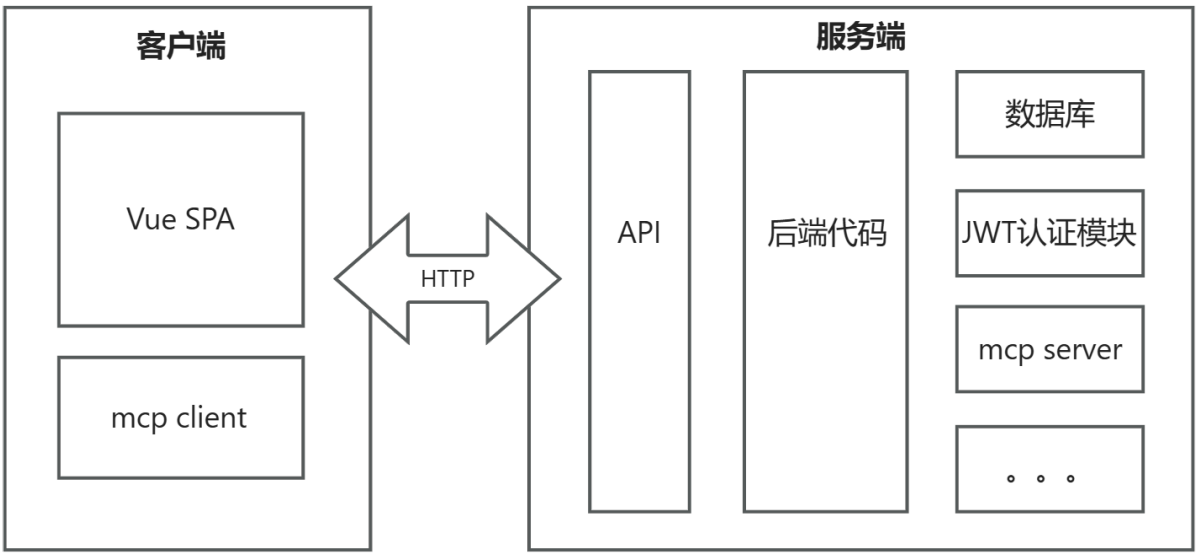
系统应具备良好的可维护性，确保长期稳定运行和持续优化。具体要求如下：

1. 模块化设计：采用清晰的模块化架构，项目结构层次分明
2. 文档完备：提供完整、清晰的技术文档，结合图表进行详细说明
3. 代码规范：严格遵循代码规范，提供充分的注释说明
4. 测试体系：建立完善的测试体系，支持单元测试、集成测试等多种测试方式
5. 版本管理：使用 Git 进行代码版本管理，确保开发、测试和发布流程的规范有序

三、总体设计

3.1 技术介绍

本项目采用前后端分离的 Web 开发架构，后端提供 RESTful API 供前端调用，同时实现了 MCP（Model Context Protocol）服务器，将前端作为 MCP 客户端，实现对 AI 智能查找图片功能的支持。系统处理流程如下图所示：



整个项目的技术选型如下：

- 前端：Vue3(TypeScript) + Element Plus 组件库 + Vue Router + npm 包管理器 + Vite 构建工具
- 后端：Gin 框架(Go) + GORM
- 数据库：MySQL + Redis
- MCP：MCP TypeScript SDK + MCP Python SDK

3.1.1 前端

本项目前端开发语言主要使用 TypeScript（TS）。TS 作为 JavaScript（ES6+）的超集，引入了静态类型系统，能够在编译阶段和编码过程中提供全面的类型检查，有效避免运行时错误，显著提升代码的可靠性与可维护性。此外，TS 与现有的 JavaScript 生态系统完全兼容，无需担心无法使用某些 JS 库的问题。

本项目前端框架选用 Vue 3。作为一款渐进式用户界面构建框架，Vue 3 的设计理念使其能够适应不同规模的前端开发需求，支持按需集成各类工具与技术。其核心特性包括高效的声明式渲染、细粒度的响应式系统以及强大的组件化能力。本项目将完全使用组合式 API（Composition API）进行开发，将特定功能的相关逻辑集中在一起，便于阅读和维护。

组件库选用 Element Plus，该组件库基于 Vue 3 构建，面向设计师与开发者，提供大量高质量、开箱即用的组件，涵盖数据展示、数据录入、导航、反馈等几乎所有后台系统所需的界面元素，是当前 Vue 3 生态中最流行、最成熟的企业级 UI 框架之一。

3.1.2 后端

本项目后端开发语言选用 Go（Golang），该语言由 Google 开源，设计之初即定位于系统级编程语言，强调高效率和高性能。Go 具备语法简洁、编译快速的特性，能够显著提升开发迭代速度。其核心优势在于对高并发的原生支持，这使得它非常适合构建高吞吐量、低延迟的 Web 服务和

微服务架构。此外，Go 语言本身内置了完备的包管理工具、高效的单元测试框架以及强大的标准库，为快速构建稳定可靠的后端服务提供了坚实的基础。

本项目后端框架选用 Gin。Gin 是一个基于 Go 语言开发的高性能、轻量级 Web 框架。它继承了 Go 语言在高并发方面的原生优势，并以其极快的路由速度和低内存占用而闻名。Gin 提供了强大的核心功能集，包括灵活的中间件机制、健壮的 HTTP 请求路由（支持路由分组）、JSON 数据绑定和验证、以及内置的 Crash 崩溃处理。这些特性结合在一起，使得 Gin 成为构建高吞吐量 API 服务和快速开发可维护性强的后端应用的理想选择。

3.1.3 数据库

关系型数据库管理系统选用 MySQL。MySQL 是最流行的关系型数据库管理系统之一，由瑞典 MySQL AB 公司开发，目前属于 Oracle 公司。本项目将使用 GORM 与 MySQL 进行交互，GORM 提供了一套高度抽象且开发者友好的 API 接口，能够简化复杂的 SQL 数据库操作，实现类型安全的数据交互，从而显著提升后端开发效率和数据库层的代码质量。

此外，本系统还将引入 Redis 作为 MySQL 的缓存层。Redis 是一个开源、内存中的键值对存储系统，凭借其极低的延迟和高并发读写能力，主要承担数据缓存、会话管理和分布式锁等核心功能。Redis 灵活的数据结构也使其能够胜任消息中间件的职能。本项目将采用 go-redis 与 Redis 进行交互，实现与 Gin 框架的高效集成。

3.1.4 认证与鉴权

本系统的认证与鉴权采用 JWT（JSON Web Token）技术。JWT 是一种定义紧凑、自包含的开放标准（RFC 7519），用于在客户端与服务端之间安全地传输信息。由于其无状态性和多平台适用性，JWT 能够简洁而高效地保障用户身份验证与权限分发过程，满足现代 Web 应用的访问控制需求。本项目将使用 `github.com/golang-jwt/jwt/v4` 来实现 JWT 的生成、解析和验证逻辑，确保认证机制的安全性和高效性。

3.1.5 第三方 API

考虑到用户交互和数据呈现的便利性，系统地址信息展示不仅使用原始的经纬度坐标。本项目选用了高德地图的逆地理编码（Reverse Geocoding）API。该 API 能够根据输入的经纬度坐标，精确解析并返回完整的国内省市区位置信息。

本系统的 AI 图像识别功能将使用阿里云视觉智能开放平台的图像分类 API 服务。该服务基于阿里云成熟的深度学习模型，能够对上传图片进行高精度、多维度的主题识别和内容分析，并自动生成精确的标签集合。

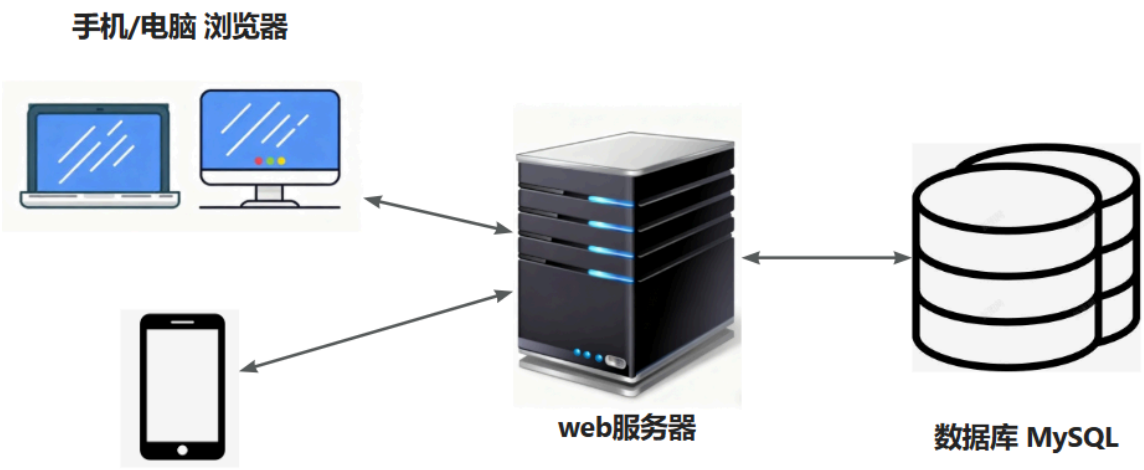
3.1.6 MCP

MCP（Model Context Protocol，模型上下文协议）是一种开放协议，旨在实现大型语言模型（LLM）应用与外部数据源、工具和服务之间的无缝集成。本项目将在前端使用 TypeScript 实现 MCP 客户端，然后使用 Python 实现 MCP 服务器，使系统能够支持用户通过对话方式查询网站图片。TypeScript 和 Python 均使用官方的 MCP SDK。

3.2 架构图

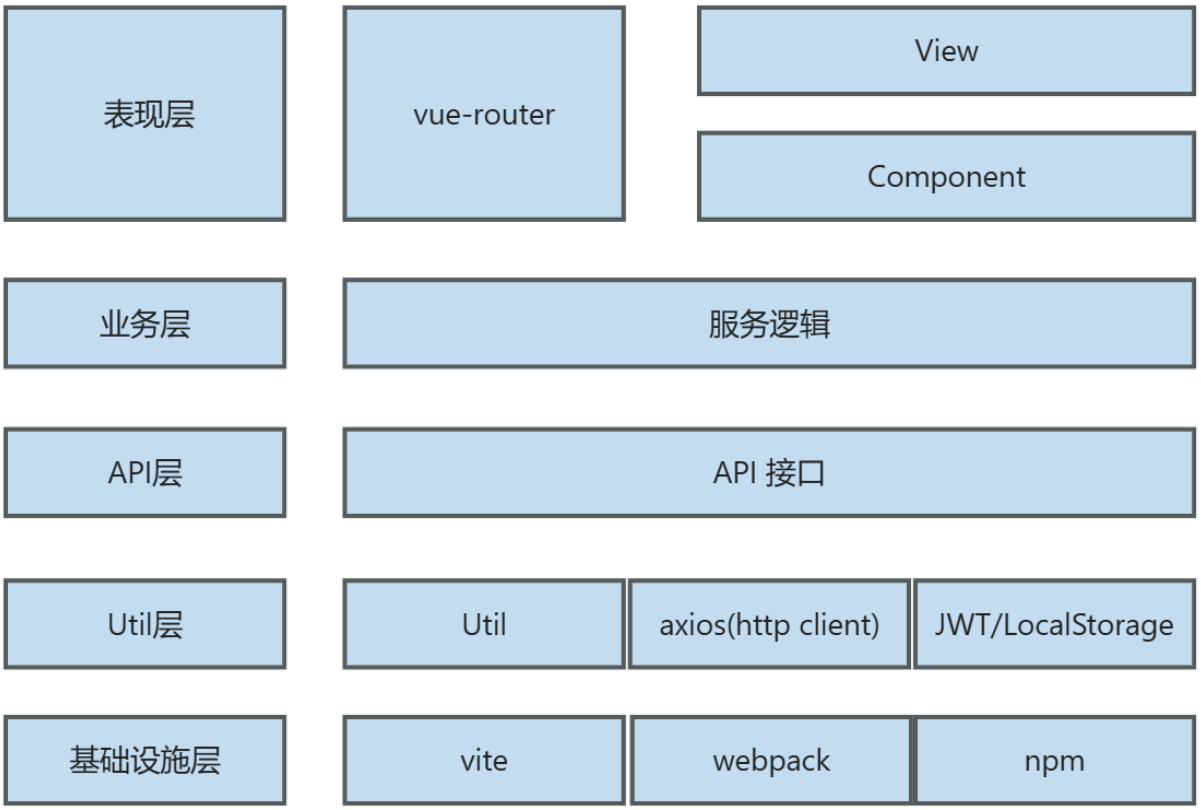
3.2.1 系统总体架构图

系统的总体架构如下图所示，其中客户端支持 PC 和手机等多种设备访问，后端服务器可以接收前端网页发出的 HTTP 请求并进行处理，将需要持久化的数据存储在数据库中。

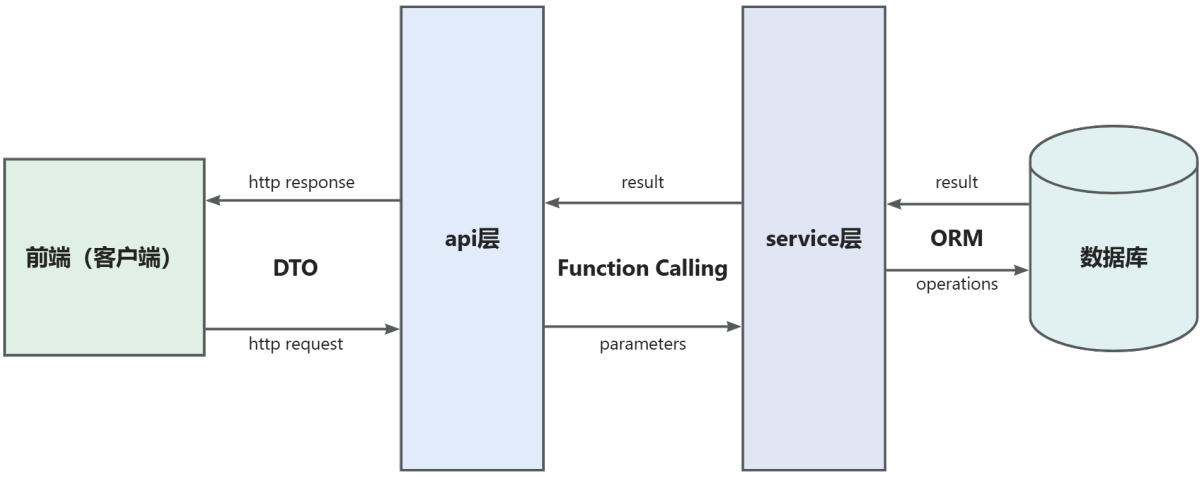


3.2.2 前端架构图

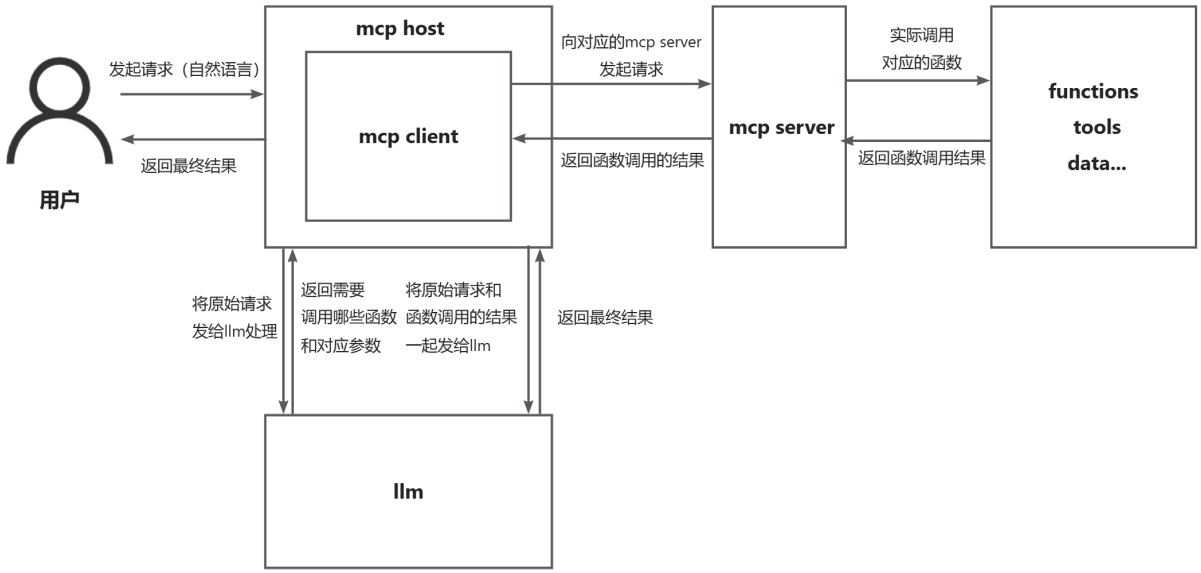
该项目前端主要采用 Vue 3 框架，并采用由饿了么团队维护的 Element Plus 作为组件库。包管理器选择 npm，打包工具选择 webpack，构建工具选择 Vite。



3.2.3 后端架构图



3.2.4 MCP 架构图



3.3 系统运行环境

系统运行环境包括硬件环境和软件环境，以下按照客户端和服务器端分别列举运行环境要求：

3.3.1 客户端

对用户端的硬件要求如下：

名称	要求
CPU	以 Windows 为例 9th COREi5 以上
内存	2GB 以上
硬盘	256GB 以上
显示器	全彩显示
键盘，鼠标	可正常使用

对客户端的软件要求如下：

- 操作系统：Windows 7 以上、Linux 或 macOS

- 浏览器：主流浏览器，如 Edge、Firefox 和 Chrome 等

3.3.2 服务器端

对服务器端的硬件要求如下：

名称	要求
CPU	以 Windows 为例 11thCOREi5 以上
内存	4GB 以上
硬盘	512GB 以上

对服务器端的软件要求如下：

类型	名称	版本
操作系统	Windows 10/11、Linux 或 macOS	
关系型数据库	MySQL	9.0 以上
key-value 数据库	Redis	8.0 以上
后端环境	Go	1.24 以上
前端环境	npm	10.0 以上

上述软件依赖均通过 Docker 部署，最终项目中会提供 Docker Compose 文件，所以服务端需要有较新版本（v25.0 以上）的 Docker。

四、数据库设计与 ER 图

4.1 数据库表设计

本系统的数据库表设计如下三张，具体说明见下方表格：

1. User(id, name, password_hash, email, is_admin, is_suspended, created_time, updated_time)
2. Image(id, name, time, longitude, latitude, province, city, district, formatted_address, width_pixel, height_pixel, user_id, created_time, updated_time)
3. Tag(id, name, image_id, created_time, updated_time)

4.1.1 User

User 表用于存储用户的基本信息。

字段名	类型	描述	备注
id	int	用户 id	主键、非空、自增
name	varchar(64)	用户名	非空，唯一
password_hash	char(60)	密码哈希	非空
email	varchar(64)	邮箱	非空，唯一
is_admin	bool	是否为管理员	非空，默认 false
is_suspended	bool	是否被封禁	非空，默认 false
created_time	datetime	创建时间	GORM 自动设置
updated_time	datetime	更新时间	GORM 自动设置

4.1.2 Image

Image 表用于存储图片的元信息。由于图片文件体积较大，不适合直接存储在数据库中，因此本系统采用文件系统存储图片文件，数据库中仅存储图片的文件名。通过图片存储目录和文件名即可定位到对应的图片文件。

字段名	类型	描述	备注
id	int	图片 id	主键、非空、自增
name	varchar(128)	图片名	非空，唯一
time	datetime	拍摄时间	可空
longitude	decimal(10, 6)	经度	可空
latitude	decimal(10, 6)	纬度	可空
province	varchar(16)	省份	可空
city	varchar(16)	市	可空
district	varchar(16)	区	可空
formatted_address	varchar(64)	完整位置	可空
width_pixel	int	宽像素	可空
height_pixel	int	高像素	可空
user_id	int	用户 id	非空，外键
created_time	datetime	创建时间	GORM 自动设置
updated_time	datetime	更新时间	GORM 自动设置

4.1.3 Tag

Tag 表用于存储图片的标签信息，实现图片的分类管理功能。

字段名	类型	描述	备注
id	int	标签 id	主键、非空、自增
name	varchar(16)	标签名	非空，唯一
image_id	int	图片 id	非空，外键
created_time	datetime	创建时间	GORM 自动设置
updated_time	datetime	更新时间	GORM 自动设置

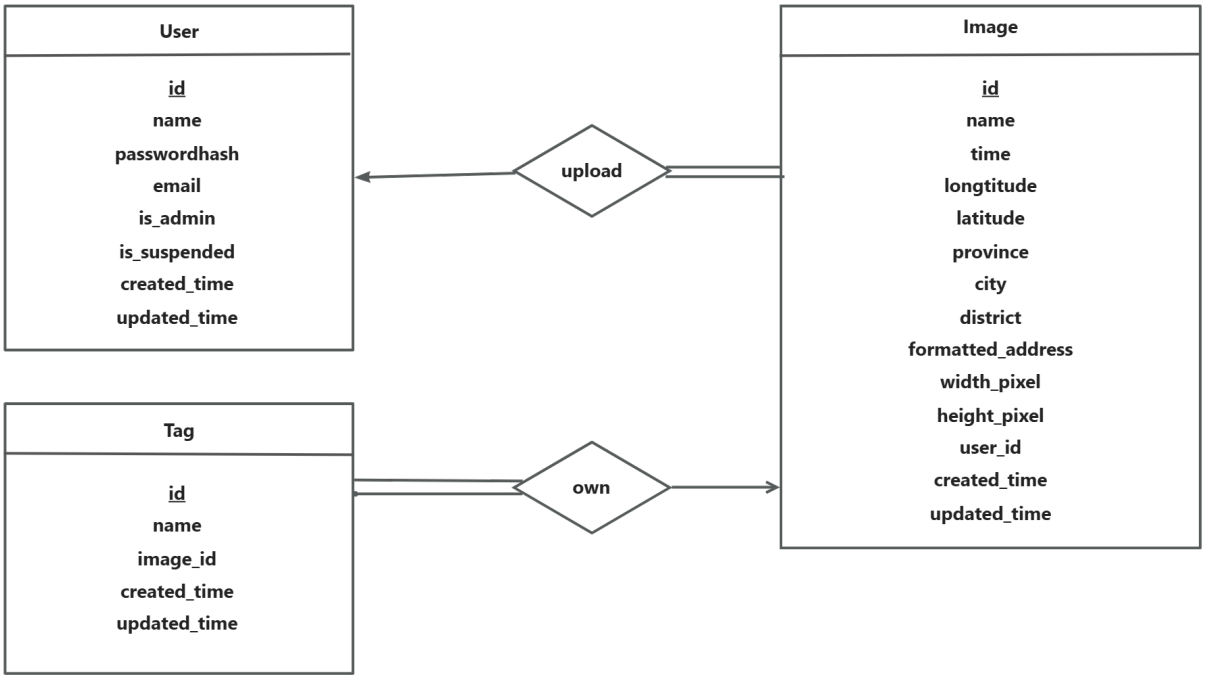
4.1.4 缩略图存储

本项目中图片存储在后端服务器，在 Gin 框架中配置静态文件目录，以便前端直接根据 URL 显示图片。本项目将每位用户的图片放在各自的文件夹（名称为用户 ID）下存储，原图和缩略图分别存储在不同的文件夹中。后端大致的目录结构如下所示：

```
server/ # 后端目录
|______ public/ # 静态文件目录
|       |______ user_id # 具体的用户ID
|               |______ origin # 原图目录
|               |______ thumbnail # 缩略图目录
|______ # 其它目录
```

由于原图和缩略图的路径都可以通过数据库中的图片元信息计算得出，因此不需要额外在数据库中存储缩略图的路径。此外，由于用户可能上传同名图片，为了避免文件覆盖，本项目会为每张图片的名称加上一个 UUID（Universally Unique Identifier）。

4.2 ER 图



五、系统接口设计

5.1 统一说明

系统接口主要用于前后端交互，本项目前端使用 axios 作为 HTTP 客户端，与后端进行异步交互，而后端则提供一系列 API 和 Service，并绑定到相应路由上。本系统的 API 设计一定程度上遵循 RESTful 规范，具体设计如下。

注意：由于目前系统仍处于开发阶段，API 接口将在编码过程中进行调整，最终的 API 接口与本文档可能存在差异。

对于入参，本项目在参数较少时选择使用 HTTP 的 query 参数，在参数较多时选择在 body 中使用 JSON 格式传输 DTO（数据传输对象）；对于出参，本项目规定统一的返回格式如下所示：

```
{
  "code": 200, // HTTP返回码
  "message": "", // 返回的消息
  "data": [] // 返回的数据，可以是任意类型
}
```

其中 code 是 HTTP 返回码，与实际返回码保持一致；message 是后端返回的提示信息，这两部分的作用比较固定；data 是后端返回的具体数据，以下接口返回参数说明中只解释 data 部分。

5.2 用户信息相关接口

用户信息类接口主要使用 User 表的数据，并对 User 表进行增删改查操作，提供的 URL 均以 user 开头，具体的 API 主要有：

5.2.1 用户注册

接口 URL	/user/register
HTTP 请求类型	POST
请求参数类型	application/json
请求参数示例	<pre>{ "username": "李华", "email": "3230106666@zju.edu.cn", "password": "123456" }</pre>
请求参数说明	username、email 和 password 分别是用户名、邮箱和密码

返回值示例	<pre>{
	  "code": 0,
	  "message": "string",
	  "data": {
	  "id": 0,
	  "username": "string",
	  "email": "string",
	  "isAdmin": true,
	  "createTime": "string",
	  "updateTime": "string"
	  }
	}</pre>

返回值说明	id 是用户 ID，username 是用户名，email 是用户邮箱，isAdmin 表示用户是否是管理员，createdTime 和 updatedTime 分别是用户的创建和修改时间
接口简介	用于用户注册，会在数据库表中添加对应信息，默认不是管理员

5.2.2 用户登录

接口 URL	/user/login
HTTP 请求类型	POST
请求参数类型	application/json
请求参数示例	<pre>{ "username": "李华", "password": "123456" }</pre>
请求参数说明	username 和 password 分别是用户名和密码

返回值示例	<pre>{
	  "code": 0,
	  "message": "string",
	  "data": {
	  "token": "string",
	  "user": {
	  "id": 0,
	  "username": "string",
	  "email": "string",
	  "isAdmin": true,</pre>

	<pre>"createTime": "string", "updateTime": "string" } } }</pre>
返回值说明	token 是生成的 JWT，user 中是用户的各项信息
接口简介	用于用户登录，验证用户输入的密码是否正确，若正确则返回 JWT 和用户信息，这两个东西将存储在浏览器 localStorage 中以便后续使用

5.2.3 根据 ID 查询用户

接口 URL	/user/getUserById
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	id:integer
请求参数说明	id 是用户 id

返回值示例	<pre>{
	"code": 0,
	"message": "string",
	"data": {
	"id": 0,
	"username": "string",
	"email": "string",
	"isAdmin": true,
	"createTime": "string",
	"updateTime": "string"
	}
	}</pre>

返回值说明	用户的各项信息
接口简介	用于根据 ID 获取用户信息，仅限管理员使用

5.2.4 查询所有用户

接口 URL	/user/getAllUsers
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	page:integer pasize:integer
请求参数说明	分页查询的页数和每页数量

返回值示例	<pre>{
	"code": 0,
	"message": "string",
	"data": {
	"token": "string",
	"user": {
	"id": 0,
	"username": "string",</pre>

	"email": "string", "isAdmin": true, "createTime": "string", "updateTime": "string" } } }
返回值说明	新的 token 和更新后的用户信息
接口简介	用于用户更新自己的个人信息

5.2.5 更新用户信息

接口 URL	/user/updateUser
HTTP 请求类型	PUT
请求参数类型	application/json

请求参数示例	{
	"id": 2,
	"username": "王华",
	"email": "222333@qq.com",
	"password": "abcdef"
	}

请求参数说明	id、username、email 和 password 分别是用户 id、新用户名、新邮箱和新密码

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"token": "string",
	"user": {
	"id": 0,
	"username": "string",
	"email": "string",
	"isAdmin": true,
	"createTime": "string",
	"updateTime": "string"
	}
	}
	}

返回值说明	token 是生成的 JWT，user 中是用户的各项信息
接口简介	用于用户登录，验证用户输入的密码是否正确，若正确则返回 JWT 和用户信息，这两个东西将存储在浏览器 localStorage 中以便后续使用

5.2.6 找回密码

接口 URL	/user/retrievePassword
HTTP 请求类型	POST
请求参数类型	application/json
请求参数示例	

	<pre>{ "new_pasword": "11223344", "id": 0, "username": "李华", "email":"3230106666@zju.edu.cn" }</pre>
请求参数说明	new_password 是新的密码，id、username 和 eamil 是对应账户的 id、用户名和邮箱
返回值示例	<pre>{ "code": 0, "message": "string", "data": null }</pre>
返回值说明	根据返回码判断是否成功
接口简介	用于用户找回自己的密码

5.2.7 授予管理员

接口 URL	/user/grantAdmin
HTTP 请求类型	PUT
请求参数类型	query
请求参数示例	id:integer
请求参数说明	id 是用户 id
返回值示例	<pre>{ "code": 0, "message": "string", "data": null }</pre>
返回值说明	根据返回码判断是否成功
接口简介	用于授予管理员权限，仅限管理员使用

5.2.8 撤销管理员

接口 URL	/user/revokeAdmin
HTTP 请求类型	PUT
请求参数类型	query
请求参数示例	id:integer
请求参数说明	id 是用户 id
返回值示例	<pre>{ "code": 0, "message": "string", "data": null }</pre>
返回值说明	根据返回码判断是否成功
接口简介	用于撤销管理员权限，仅限管理员使用

5.2.9 封禁用户

接口 URL	/user/suspend
HTTP 请求类型	POST
请求参数类型	query
请求参数示例	id:integer duration:integer
请求参数说明	id 是用户 id，duration 是持续时间（按分钟计算）

返回值示例	{
	"code": 0,
	"message": "string",
	"data": null
	}

返回值说明	根据返回码判断是否成功
接口简介	用于封禁一个用户一定时长，仅限管理员使用

5.2.10 解封用户

接口 URL	/user/unsuspend
HTTP 请求类型	POST
请求参数类型	query
请求参数示例	id:integer
请求参数说明	id 是待解封用户 id

返回值示例	{
	"code": 0,
	"message": "string",
	"data": null
	}

返回值说明	根据返回码判断是否成功
接口简介	用于提前解封一个被封禁了的账户，仅限管理员使用

5.3 图片类接口

图片类接口主要使用 Image 和 Tag 表的数据，并对这两张表进行增删改查操作，提供的 URL 均以 image 开头，具体的 API 主要有：

5.3.1 上传图片

接口 URL	/image/uploadImage
HTTP 请求类型	POST
请求参数类型	multipart/form-data
请求参数示例	image:string<binary> imageName:string
请求参数说明	image 是图片的二进制数据, imageName 是自定义图片名称
返回值示例	

	<pre>{ "code": 0, "message": "string", "data": null }</pre>
返回值说明	根据状态码判断是否成功
接口简介	用于上传图片，后端会将图片通过文件系统保存到对应目录，图片 EXIF 信息和其他元信息会被保存到数据库中

5.3.2 设置标签

接口 URL	/image/setTags
HTTP 请求类型	PUT
请求参数类型	application/json
请求参数示例	<pre>{ "imageID": 0, "tags": [] }</pre>
请求参数说明	imageID 是图片 ID，tags 是想要添加的标签（字符串数组）
返回值示例	<pre>{ "code": 0, "message": "string", "data": null }</pre>
返回值说明	根据状态码判断是否成功
接口简介	用于为指定图片设置标签，采用直接覆盖方式，所以增删标签均使用这个接口

5.3.3 更新图片

接口 URL	/image/updateImage
HTTP 请求类型	POST
请求参数类型	multipart/form-data
请求参数示例	image:string<binary> imageID:integer
请求参数说明	image 是新的图片的二进制数据，imageID 是图片 ID
返回值示例	<pre>{ "code": 0, "message": "string", "data": null }</pre>
返回值说明	根据状态码判断是否成功
接口简介	用于更新图片，主要用于用户在前端编辑图片后的更新操作

5.3.4 删除图片

接口 URL	/image/deleteImage
HTTP 请求类型	DELETE
请求参数类型	query
请求参数示例	imageID:integer userID:integer
请求参数说明	imageID 是图片 ID，userID 是图片拥有者的用户 ID

返回值示例	{
	"code": 0,
	"message": "string",
	"data": null
	}

返回值说明	根据状态码判断是否成功
接口简介	用于删除图片

5.3.5 查询所有图片

接口 URL	/image/getAllImages
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	page:integer pageSize:integer
请求参数说明	分页查询的页数和每页数量

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"images": [
	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}
	],
	"pageTotal": 0
	}
	}

返回值说明	总页数+图片核心信息
接口简介	用于分页查询所有图片，仅限管理员使用

5.3.6 查询个人所有图片

接口 URL	/image/getImageByID
HTTP 请求类型	GET
请求参数类型	query

请求参数示例	page:integer pageSize:integer
请求参数说明	分页查询的页数和每页数量

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"images": [
	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}
	],
	"pageTotal": 0
	}
	}

返回值说明	总页数+图片核心信息
接口简介	用于分页查询用户自己的所有图片

5.3.7 根据标签查询图片

接口 URL	/image/getImageByTag
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	page:integer pageSize:integer tag:string
请求参数说明	分页查询的页数和每页数量，tag 是查询的标签

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"images": [
	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}
	],
	"pageTotal": 0
	}
	}

返回值说明	总页数+图片核心信息
接口简介	用于分页根据 tag 查询用户自己的所有图片

5.3.8 根据时间区间查询图片

接口 URL	/image/getImageByTimeRange
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	page:integer pageSize:integer startTime:string endTime:string
请求参数说明	分页查询的页数和每页数量，以及时间区间的始末

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"images": [
	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}
	],
	"pageTotal": 0
	}
	}

返回值说明	总页数+图片核心信息
接口简介	用于分页根据时间区间查询用户自己的所有图片

5.3.9 根据省份获取图片

接口 URL	/image/getImageByProvince
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	page:integer pageSize:integer province:string
请求参数说明	分页查询的页数和每页数量

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"images": [
	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}
	],

	<pre>"pageTotal": 0 } }</pre>
返回值说明	总页数+图片核心信息
接口简介	用于根据省份分页查询用户自己的所有图片，直辖市算作省

5.3.10 根据城市查询图片

接口 URL	/image/getImageByCity
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	page:integer pageSize:integer district:string
请求参数说明	分页查询的页数和每页数量

返回值示例	<pre>{
	"code": 0,
	"message": "string",
	"data": {
	"images": [
	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}
	],
	"pageTotal": 0
	}
	}</pre>

返回值说明	总页数+图片核心信息
接口简介	用于根据城市分页查询用户自己的所有图片

5.3.11 根据区县查询图片

接口 URL	/image/getImageByDistrict
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	page:integer pageSize:integer district:string
请求参数说明	分页查询的页数和每页数量

返回值示例	<pre>{
	"code": 0,
	"message": "string",
	"data": {
	"images": [</pre>

	{
	"image_id": 0,
	"image_name": "string",
	"image_path": "string",
	"tags": []
	}
	],
	"pageTotal": 0
	}
	}

返回值说明	总页数+图片核心信息
接口简介	用于根据区县分页查询用户自己的所有图片

5.3.12 查询图片详情

接口 URL	/image/getImageDetail
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	imageID:integer
请求参数说明	图片 ID

返回值示例	{
	"code": 0,
	"message": "string",
	"data": {
	"id": 0,
	"name": "string",
	"path": "string",
	"captured_time": "string",
	"latitude": 0,
	"longitude": 0,
	"formatted_address": "string",
	"province": "string",
	"city": "string",
	"district": "string",
	"width_pixel": 0,
	"height_pixel": 0,
	"user_id": 0,
	"user": {
	"id": 0,
	"username": "string",
	"email": "string",
	"isAdmin": true,
	"createTime": "string",
	"updateTime": "string"
	},
	"created_time": "string",
	"updated_time": "string"
	}
	}

返回值说明	图片的详细信息，包括标签和 EXIF 信息等
接口简介	用于根据图片 ID 查询图片的详细信息

5.3.13 获取图片文件

接口 URL	/image/getImageFile
HTTP 请求类型	GET
请求参数类型	query
请求参数示例	imageID:integer
请求参数说明	图片 ID
返回值示例	string<binary>
返回值说明	真正的图片文件
接口简介	用于获取图片文件，主要是下载图片时需要

六、界面原型

以下是项目的界面原型，部分界面可能还比较简单，主要展示核心交互逻辑，后续会进行美化优化。

6.1 注册界面

在这个界面用户可以填写自己的基本信息，包括用户名、邮箱和密码，然后注册自己的账户。如果注册成功（用户名和邮箱都不重复，并且密码符合要求），系统会自动跳转到登录页面。



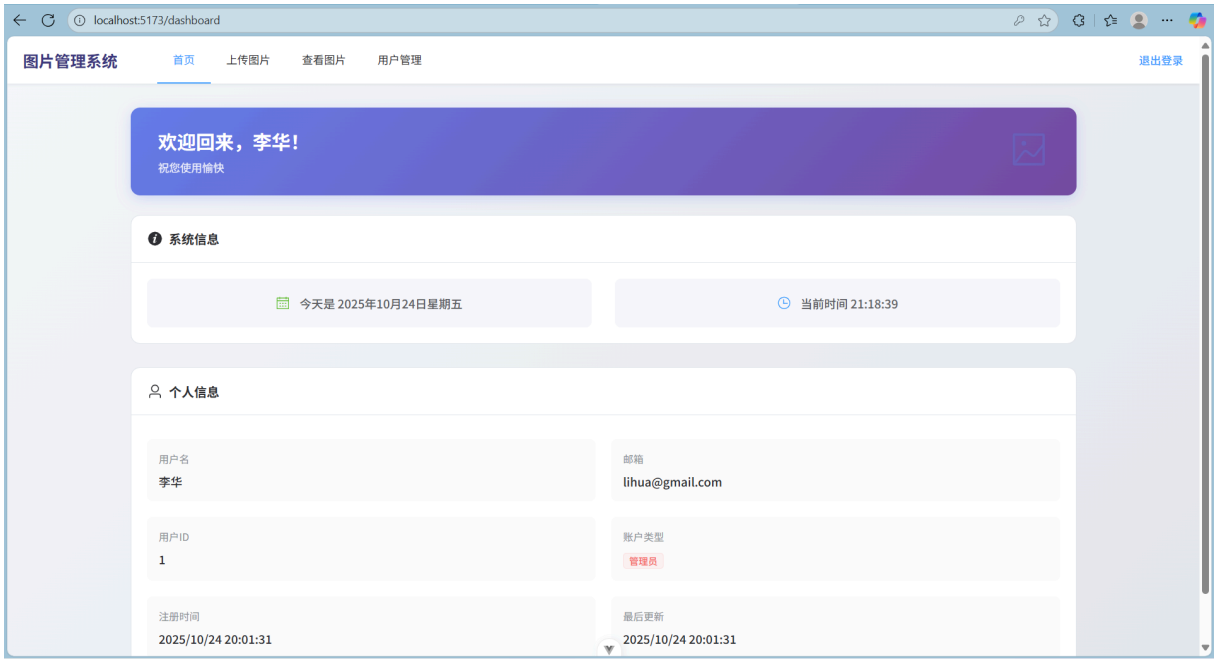
6.2 登录界面

在这个界面用户可以输入自己的用户名和密码，然后点击登录按钮。如果登录成功（密码正确，且账号未被封禁），则跳转到网站首页。如果其他页面产生了 HTTP 状态码为 401 Unauthorized 的请求（说明未登录或者账号被封禁），则会被重定向到登录页面。



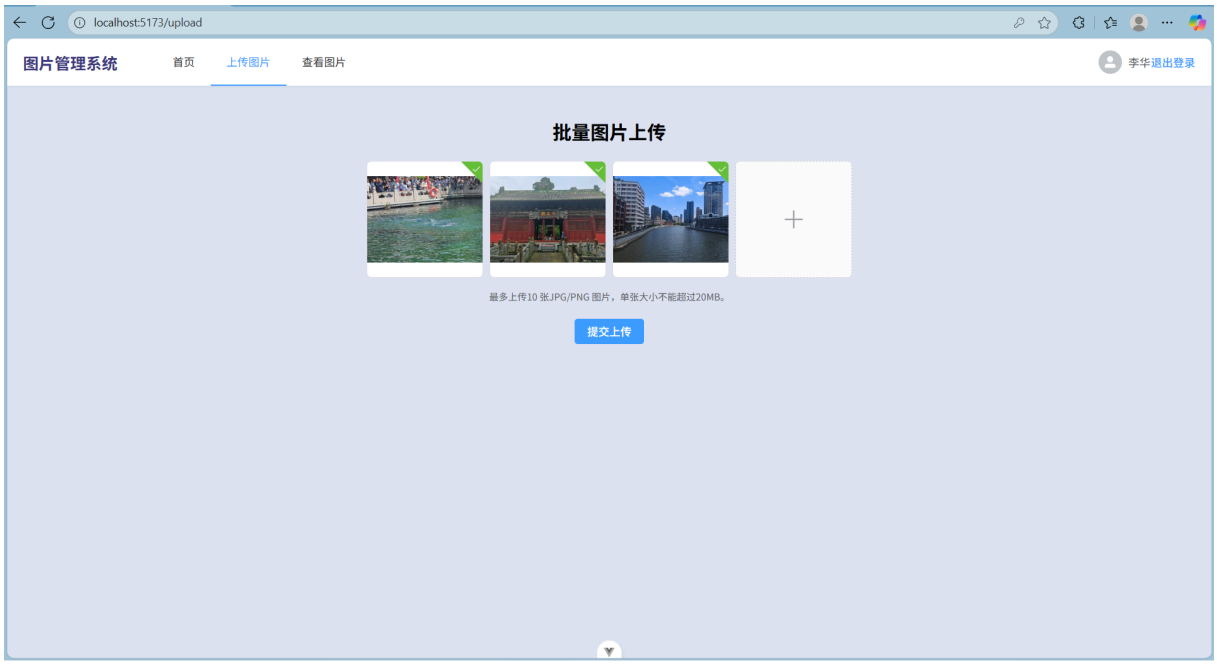
6.3 网站首页

在这个界面会显示一些系统信息和用户的个人信息，可以点击上边栏跳转到其他页面。上边栏中的“用户管理”只会在管理员的前端渲染，并且在通过上边栏跳转时也会进行权限验证。



6.4 上传图片界面

这个页面用于上传图片，支持批量上传（一次最多 10 张）。点击加号方框即可从本地挑选图片上传，已经上传到前端的图片会在前端显示预览图，点击提交上传按钮才会正式将图片发送到后端存储。

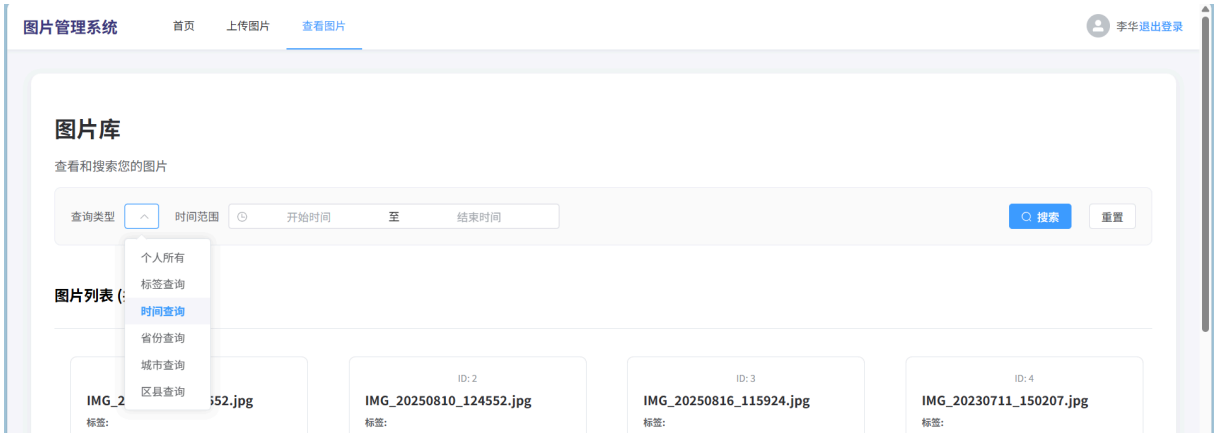


6.5 查看图片界面

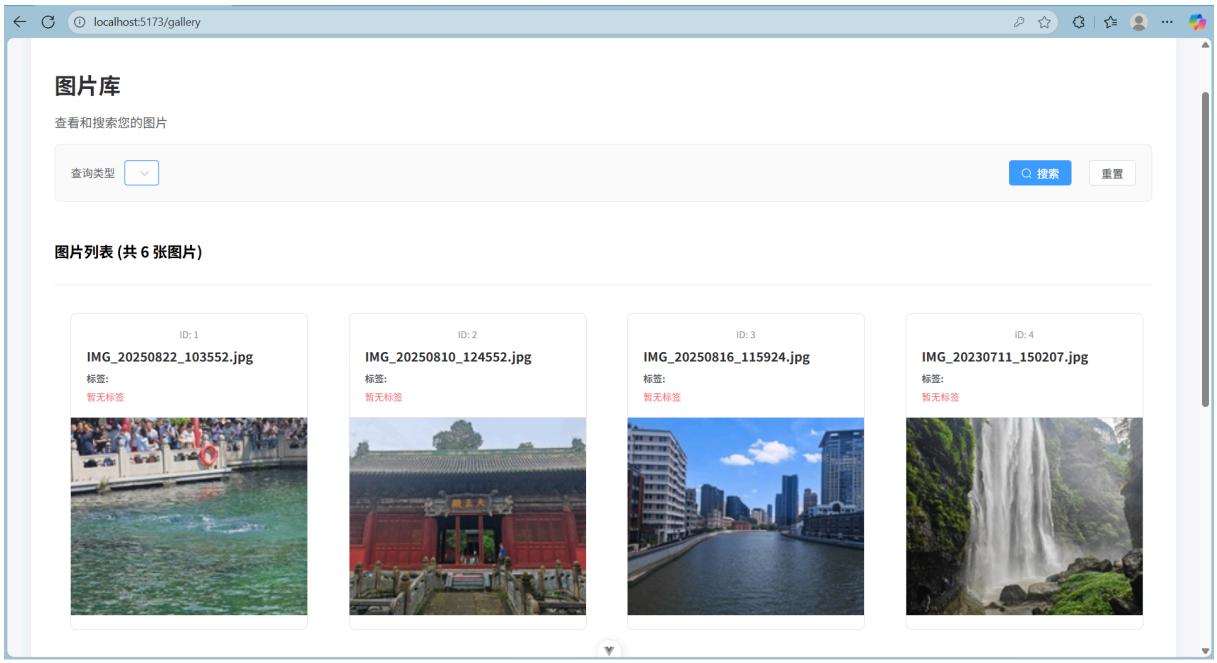
这个界面的功能比较丰富，下面分别介绍各个功能：

可以在这个界面按照不同条件查询图片，支持的查询条件包括：

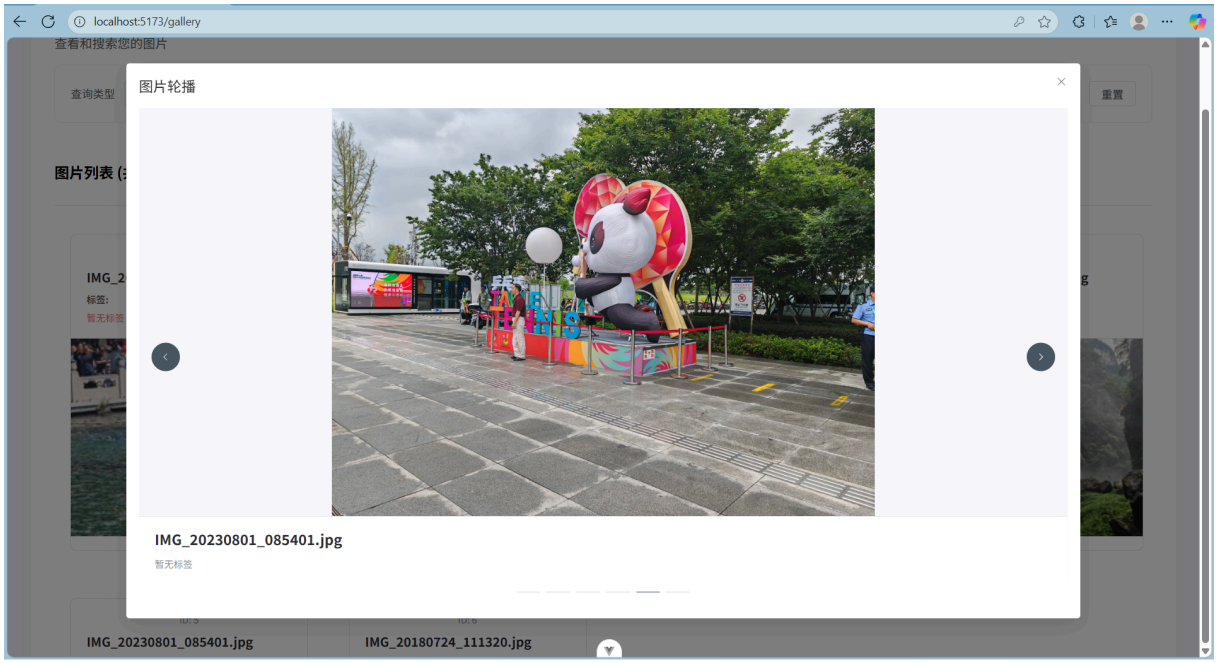
- 个人所有图片
- 图片标签
- 拍摄时间区间
- 省市区地理位置



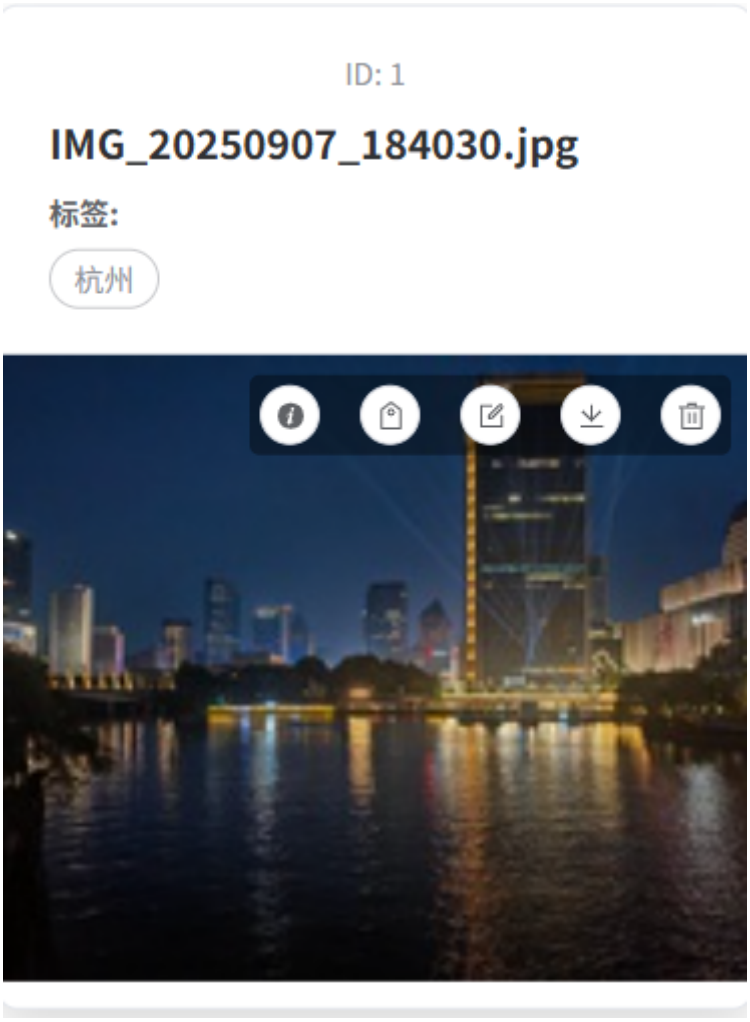
查询结果如下图所示，图片会依次排列并显示最核心的信息，即图片 ID、图片名和标签。



点击图片可以进行图片轮播，并且会自动切换到被点击的图片。



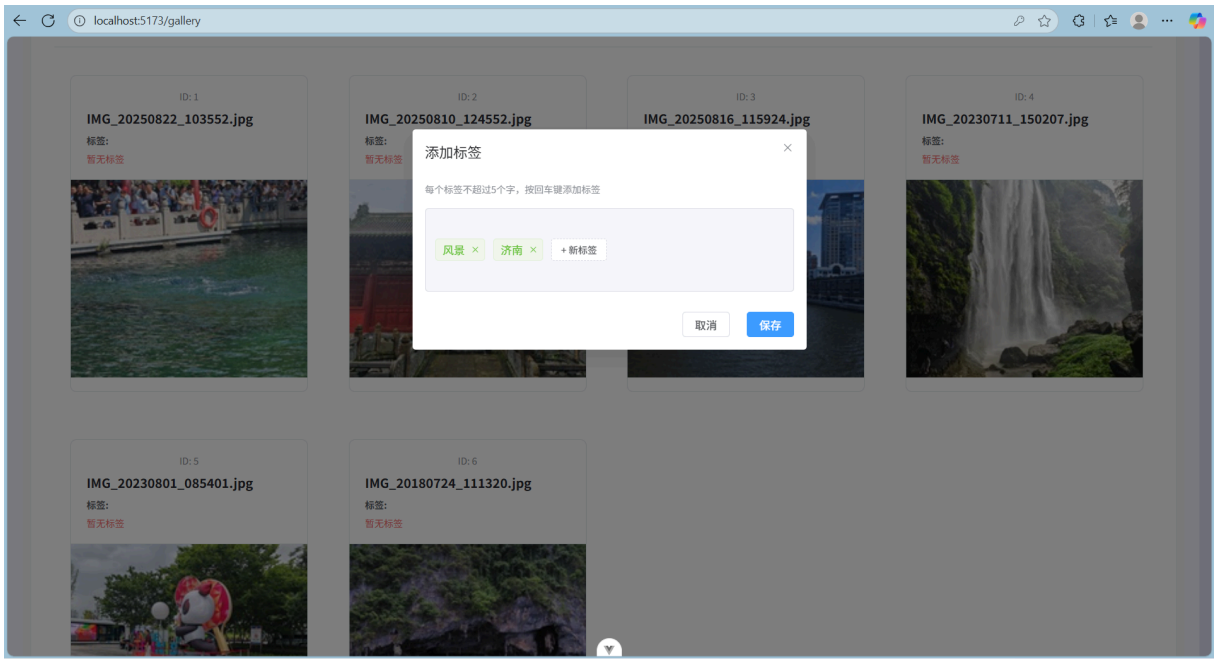
当鼠标悬停在图片上时，图片右上角会浮现四个图标，分别对应查看图片详情、添加标签、编辑图片、下载图片和删除图片。



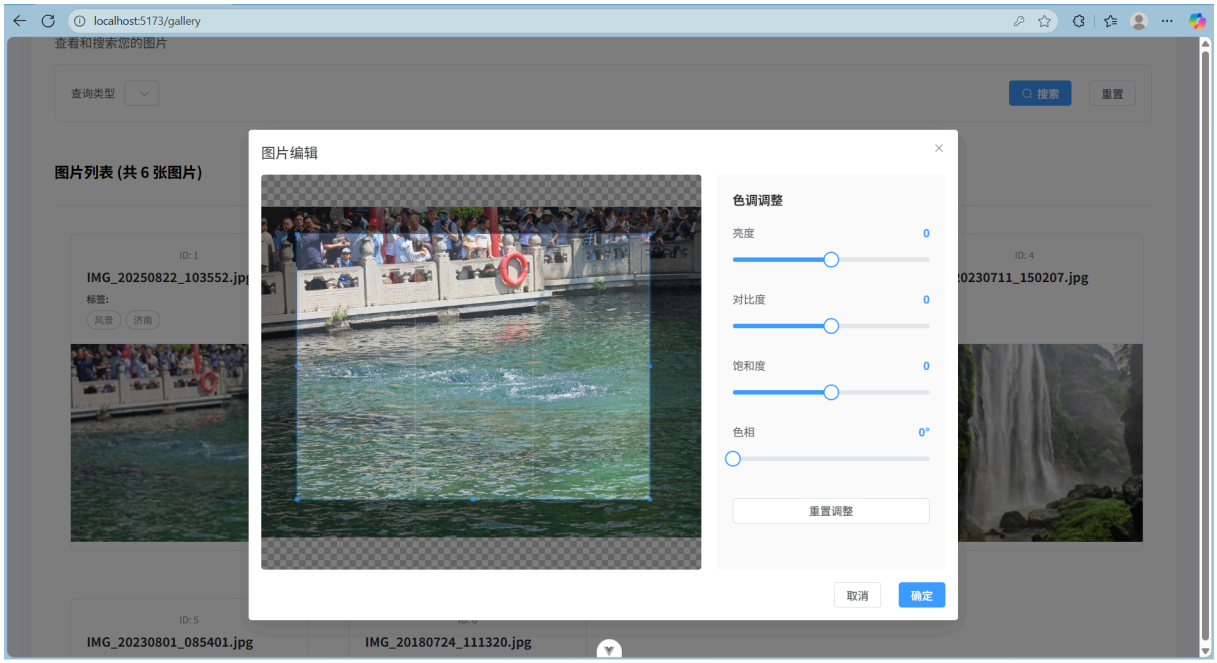
点击查看图片详情图标后，会弹出对话框显示图片原图和详细信息（主要是图片的 EXIF 信息）。



点击添加标签图标后，会弹出对话框显示当前图片的标签，可以添加新的标签，也可以删除已有的标签。



点击编辑图片图标后，会弹出对话框显示当前图片原图，并且可以裁剪图片，调整图片的亮度、对比度、饱和度和色相等属性。



点击下载图片图标后，图片原始文件会被发送到前端，然后自动在浏览器中下载。点击删除图片图标后，如果确认删除，则图片文件和元信息将分别从文件系统和数据库中删除。

七、系统出错设计

7.1 出错信息

系统通过统一的错误码和友好的信息反馈异常情况，关键错误方式及处理方式如下：

错误类型	错误信息提示	可能原因	处理方法
后端连接失败	弹窗提示“系统繁忙, 请稍后再试”	1. 后端启动失败 2. ip 配置错误	1. 检查配置文件 2. 重启服务
权限不足	页面提示“您无权限执行此操作”	普通用户视图越权访问管理员功能	记录日志并跳转至默认页面
未登录直接访问业务页面	弹窗提示“您尚未登录”	直接通过 URL 访问业务页面	自动返回登录页面
已在业务页面操作但 JWT 无效	弹窗提示“您已退出登录”	长时间未操作导致 JWT 过期	自动返回登录页面
访问不存在的页面	弹窗提示“您访问的页面不存在”	URL 输入了错误地址	跳转到 404 页面
文件格式错误	提示“图片格式仅支持 png 和 jpg”	上传其它格式的图片或信息缺损的图片	强制重新选择图片

7.2 补救措施

本项目主要通过系统日志来监控项目状态:

1. 在日志中记录所有 HTTP 请求的信息，包含发起操作者 IP、时间戳及详细参数
2. 日志存储在后端文件系统中，保留周期为 2 个月

八、附录

8.1 项目进度安排

项目的进度安排如下，目前正在稳步推进中:

时间	任务
2025 年 9 月	项目初期分析和调研
2025 年 10 月	完成设计报告
2025 年 11 月	基本完成项目编码
2025 年 12 月	调试和完善项目，并完成测试报告和产品说明书

8.2 备注

本设计报告为项目初期的评估，后续还会根据实际情况进行调整。